

SQL – Test

Q1. List top 5 customers by total order amount.

Retrieve the top 5 customers who have spent the most across all sales orders. Show CustomerID, CustomerName, and TotalSpent.

SELECT TOP 5

c.CustomerID,

c.Name AS CustomerName,

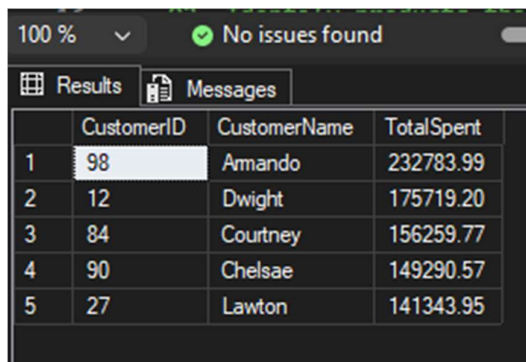
SUM(so.TotalAmount) AS TotalSpent

FROM customer c

JOIN salesorder so ON c.CustomerID = so.CustomerID

GROUP BY c.CustomerID, c.Name

ORDER BY TotalSpent DESC;



The screenshot shows a SQL Server query results window. At the top, there is a status bar with '100 %' and a green checkmark indicating 'No issues found'. Below this are two tabs: 'Results' and 'Messages'. The 'Results' tab is active, displaying a table with the following data:

	CustomerID	CustomerName	TotalSpent
1	98	Armando	232783.99
2	12	Dwight	175719.20
3	84	Courtney	156259.77
4	90	Chelsae	149290.57
5	27	Lawton	141343.95

--Q2. Find the number of products supplied by each supplier.

--Display SupplierID, SupplierName, and ProductCount. Only include suppliers that have more than 10 products.

SELECT

s.SupplierID,

s.Name AS SupplierName,

COUNT(DISTINCT pod.ProductID) AS ProductCount

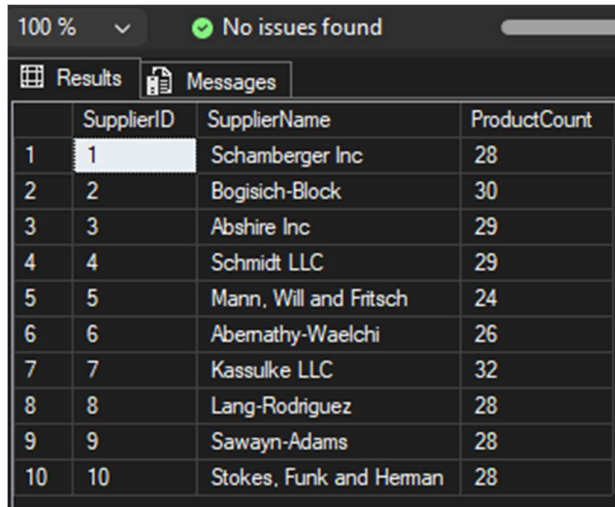
FROM supplier s

JOIN purchaseorder po ON s.SupplierID = po.SupplierID

JOIN purchaseorderdetail pod ON po.OrderID = pod.OrderID

GROUP BY s.SupplierID, s.Name

HAVING COUNT(DISTINCT pod.ProductID) > 10;



100 % No issues found

	SupplierID	SupplierName	ProductCount
1	1	Schamberger Inc	28
2	2	Bogisich-Block	30
3	3	Abshire Inc	29
4	4	Schmidt LLC	29
5	5	Mann, Will and Fritsch	24
6	6	Abemathy-Waelchi	26
7	7	Kassulke LLC	32
8	8	Lang-Rodriguez	28
9	9	Sawayn-Adams	28
10	10	Stokes, Funk and Herman	28

Q3. Identify products that have been ordered but never returned.

Show ProductID, ProductName, and total order quantity.

SELECT

p.ProductID,

p.Name AS ProductName,

SUM(sod.Quantity) AS TotalOrderQuantity

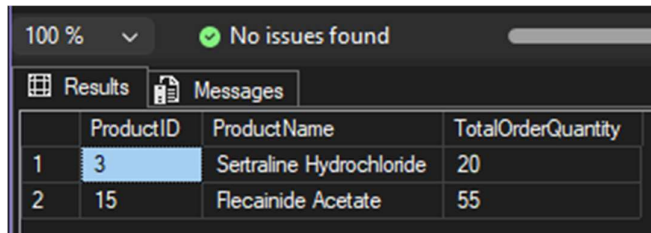
FROM product p

JOIN salesorderdetail sod ON p.ProductID = sod.ProductID

LEFT JOIN ReturnDetail rd ON p.ProductID = rd.ProductID

WHERE rd.ProductID IS NULL

GROUP BY p.ProductID, p.Name;



100 % No issues found

	ProductID	ProductName	TotalOrderQuantity
1	3	Sertraline Hydrochloride	20
2	15	Flecainide Acetate	55

Q4. For each category, find the most expensive product.

Display CategoryID, CategoryName, ProductName, and Price. Use a subquery to get the max price per category.

```
SELECT
```

```
    c.CategoryID,
```

```
    c.Name AS CategoryName,
```

```
    p.Name AS ProductName,
```

```
    p.Price
```

```
FROM category c
```

```
JOIN product p ON c.CategoryID = p.CategoryID
```

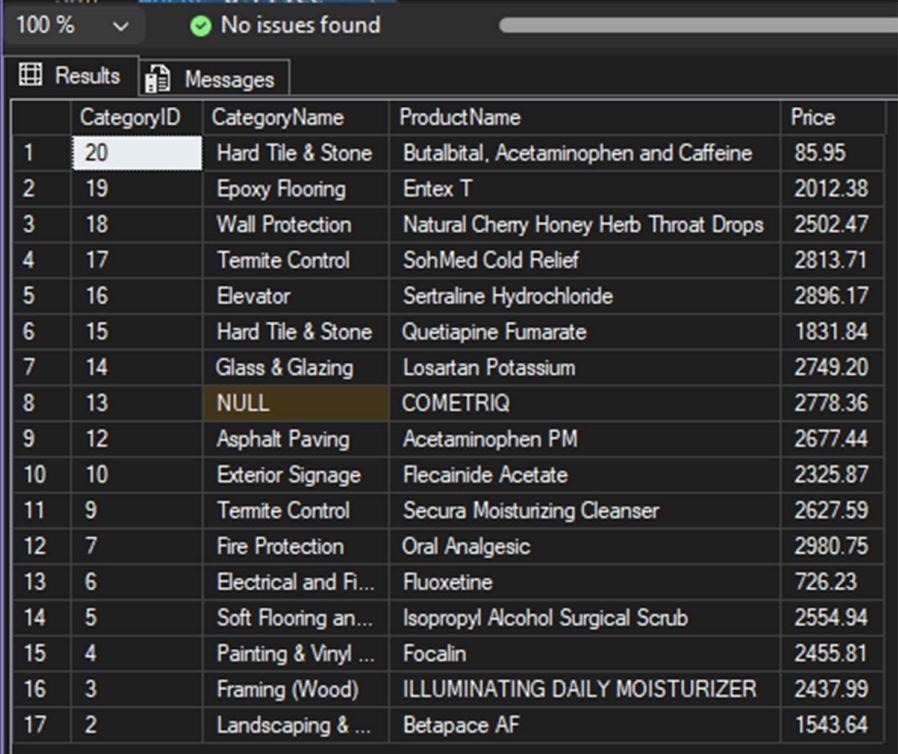
```
WHERE p.Price = (
```

```
    SELECT MAX(subP.Price)
```

```
    FROM product subP
```

```
    WHERE subP.CategoryID = c.CategoryID
```

```
);
```



	CategoryID	CategoryName	ProductName	Price
1	20	Hard Tile & Stone	Butalbitol, Acetaminophen and Caffeine	85.95
2	19	Epoxy Flooring	Entex T	2012.38
3	18	Wall Protection	Natural Cherry Honey Herb Throat Drops	2502.47
4	17	Termite Control	SohMed Cold Relief	2813.71
5	16	Elevator	Sertraline Hydrochloride	2896.17
6	15	Hard Tile & Stone	Quetiapine Fumarate	1831.84
7	14	Glass & Glazing	Losartan Potassium	2749.20
8	13	NULL	COMETRIQ	2778.36
9	12	Asphalt Paving	Acetaminophen PM	2677.44
10	10	Exterior Signage	Flecainide Acetate	2325.87
11	9	Termite Control	Secura Moisturizing Cleanser	2627.59
12	7	Fire Protection	Oral Analgesic	2980.75
13	6	Electrical and Fi...	Fluoxetine	726.23
14	5	Soft Flooring an...	Isopropyl Alcohol Surgical Scrub	2554.94
15	4	Painting & Vinyl ...	Focalin	2455.81
16	3	Framing (Wood)	ILLUMINATING DAILY MOISTURIZER	2437.99
17	2	Landscaping & ...	Betapace AF	1543.64

--Q5. List all sales orders with customer name, product name, category, and supplier.

--For each sales order, display:

--OrderID, CustomerName, ProductName, CategoryName, SupplierName, and Quantity.

SELECT

so.OrderID,

c.Name AS CustomerName,

p.Name AS ProductName,

cat.Name AS CategoryName,

s.Name AS SupplierName,

sod.Quantity

FROM salesorder so

JOIN customer c ON so.CustomerID = c.CustomerID

JOIN salesorderdetail sod ON so.OrderID = sod.OrderID

JOIN product p ON sod.ProductID = p.ProductID

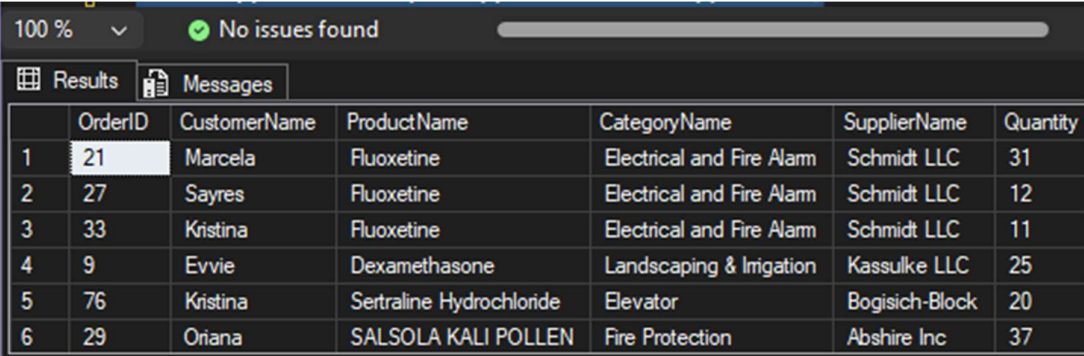
JOIN category cat ON p.CategoryID = cat.CategoryID

-- Connecting back to supplier via purchase order details to find who supplies it

JOIN purchaseorderdetail pod ON p.ProductID = pod.ProductID

JOIN purchaseorder po ON pod.OrderID = po.OrderID

JOIN supplier s ON po.SupplierID = s.SupplierID;



	OrderID	CustomerName	ProductName	CategoryName	SupplierName	Quantity
1	21	Marcela	Fluoxetine	Electrical and Fire Alarm	Schmidt LLC	31
2	27	Sayres	Fluoxetine	Electrical and Fire Alarm	Schmidt LLC	12
3	33	Kristina	Fluoxetine	Electrical and Fire Alarm	Schmidt LLC	11
4	9	Evvie	Dexamethasone	Landscaping & Irigation	Kassulke LLC	25
5	76	Kristina	Sertraline Hydrochloride	Elevator	Bogisich-Block	20
6	29	Oriana	SALSOLA KALI POLLEN	Fire Protection	Abshire Inc	37

Q6. Find all shipments with details of warehouse, manager, and products shipped.

Display:

ShipmentID, WarehouseName, ManagerName, ProductName, QuantityShipped, and TrackingNumber.

SELECT

s.ShipmentID,

w.WarehouseID,

e.Name AS ManagerName,

p.Name AS ProductName,

sd.Quantity AS QuantityShipped,

s.TrackingNumber

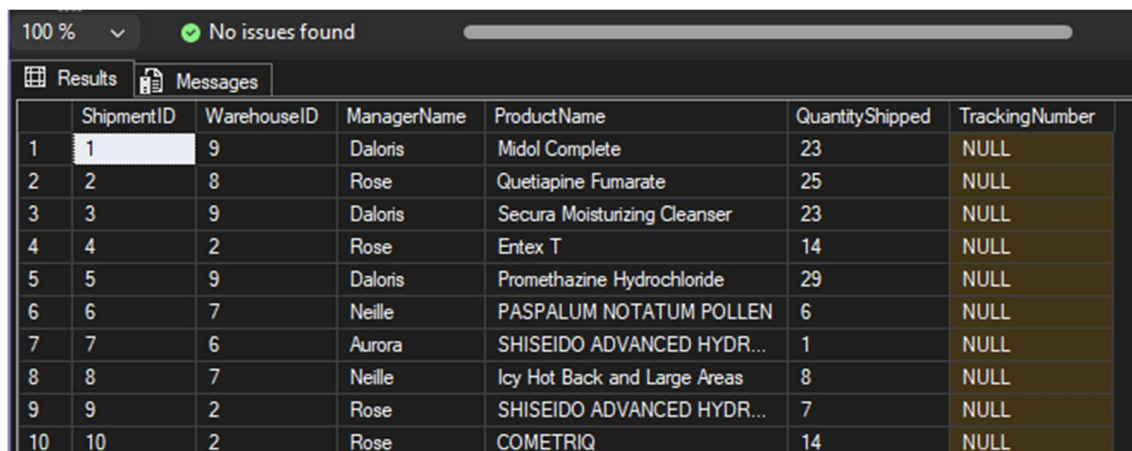
FROM shipment s

JOIN warehouse w ON s.WarehouseID = w.WarehouseID

JOIN employee e ON w.ManagerID = e.EmployeeID

JOIN shipmentdetail sd ON s.ShipmentID = sd.ShipmentID

JOIN product p ON sd.ProductID = p.ProductID;



The screenshot shows a database query results window with a dark theme. At the top, there is a status bar with '100 %' and a green checkmark icon followed by 'No issues found'. Below this are two tabs: 'Results' (active) and 'Messages'. The main area displays a table with 7 columns: ShipmentID, WarehouseID, ManagerName, ProductName, QuantityShipped, and TrackingNumber. The table contains 10 rows of data, numbered 1 through 10 in the first column. The 'TrackingNumber' column for all rows contains the value 'NULL'.

	ShipmentID	WarehouseID	ManagerName	ProductName	QuantityShipped	TrackingNumber
1	1	9	Daloris	Midol Complete	23	NULL
2	2	8	Rose	Quetiapine Fumarate	25	NULL
3	3	9	Daloris	Secura Moisturizing Cleanser	23	NULL
4	4	2	Rose	Entex T	14	NULL
5	5	9	Daloris	Promethazine Hydrochloride	29	NULL
6	6	7	Neille	PASPALUM NOTATUM POLLEN	6	NULL
7	7	6	Aurora	SHISEIDO ADVANCED HYDR...	1	NULL
8	8	7	Neille	Icy Hot Back and Large Areas	8	NULL
9	9	2	Rose	SHISEIDO ADVANCED HYDR...	7	NULL
10	10	2	Rose	COMETRIQ	14	NULL

Q7. Find the top 3 highest-value orders per customer using RANK(). Display CustomerID, CustomerName, OrderID, and TotalAmount.

WITH RankedOrders AS (

SELECT

c.CustomerID,

c.Name AS CustomerName,

so.OrderID,

so.TotalAmount,

RANK() OVER (PARTITION BY c.CustomerID ORDER BY so.TotalAmount DESC) as
OrderRank

FROM customer c

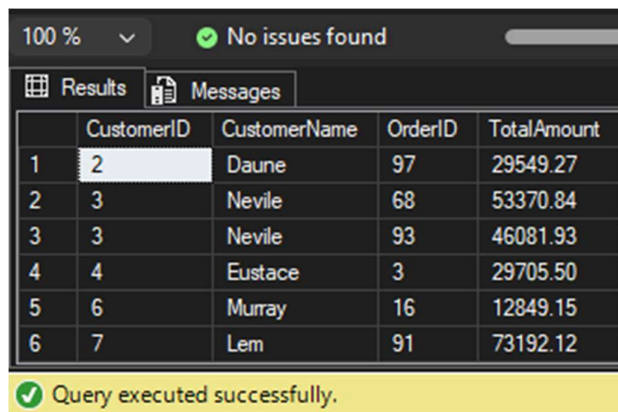
JOIN salesorder so ON c.CustomerID = so.CustomerID

)

SELECT CustomerID, CustomerName, OrderID, TotalAmount

FROM RankedOrders

WHERE OrderRank <= 3;



The screenshot shows a SQL query results window with a dark theme. At the top, there is a zoom level of 100% and a status bar indicating 'No issues found'. Below this, there are two tabs: 'Results' and 'Messages'. The 'Results' tab is active, displaying a table with 5 columns: an index column, CustomerID, CustomerName, OrderID, and TotalAmount. The table contains 6 rows of data. A yellow status bar at the bottom indicates 'Query executed successfully.'

	CustomerID	CustomerName	OrderID	TotalAmount
1	2	Daune	97	29549.27
2	3	Nevile	68	53370.84
3	3	Nevile	93	46081.93
4	4	Eustace	3	29705.50
5	6	Murray	16	12849.15
6	7	Lem	91	73192.12

Q8. For each product, show its sales history with the previous and next sales quantities (based on order date). Display ProductID, ProductName, OrderID, OrderDate, Quantity, PrevQuantity, and NextQuantity.

SELECT

p.ProductID,

p.Name AS ProductName,

so.OrderID,

so.OrderDate,

sod.Quantity,

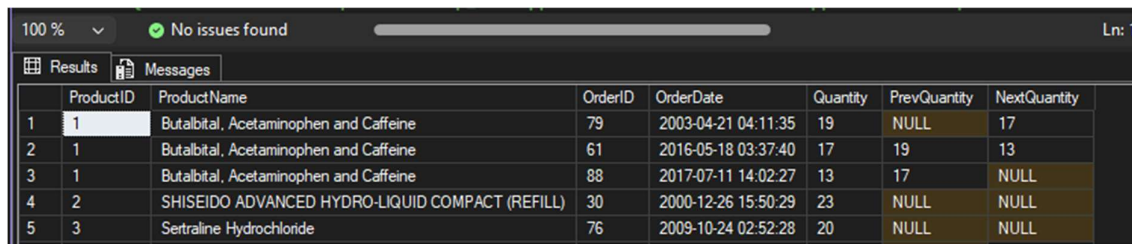
LAG(sod.Quantity) OVER (PARTITION BY p.ProductID ORDER BY so.OrderDate,
so.OrderID) AS PrevQuantity,

LEAD(sod.Quantity) OVER (PARTITION BY p.ProductID ORDER BY so.OrderDate,
so.OrderID) AS NextQuantity

FROM product p

JOIN salesorderdetail sod ON p.ProductID = sod.ProductID

JOIN salesorder so ON sod.OrderID = so.OrderID;



	ProductID	ProductName	OrderID	OrderDate	Quantity	PrevQuantity	NextQuantity
1	1	Butalbital, Acetaminophen and Caffeine	79	2003-04-21 04:11:35	19	NULL	17
2	1	Butalbital, Acetaminophen and Caffeine	61	2016-05-18 03:37:40	17	19	13
3	1	Butalbital, Acetaminophen and Caffeine	88	2017-07-11 14:02:27	13	17	NULL
4	2	SHISEIDO ADVANCED HYDRO-LIQUID COMPACT (REFILL)	30	2000-12-26 15:50:29	23	NULL	NULL
5	3	Sertraline Hydrochloride	76	2009-10-24 02:52:28	20	NULL	NULL

Q9. Create a view named vw_CustomerOrderSummary that shows for each customer: CustomerID, CustomerName, TotalOrders, TotalAmountSpent, and LastOrderDate.

CREATE VIEW vw_CustomerOrderSummary AS

SELECT

c.CustomerID,

c.Name AS CustomerName,

COUNT(so.OrderID) AS TotalOrders,

```

SUM(so.TotalAmount) AS TotalAmountSpent,

MAX(so.OrderDate) AS LastOrderDate

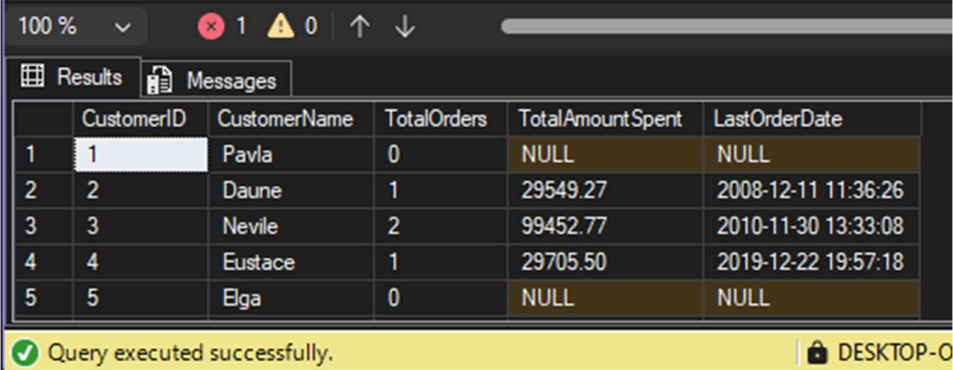
FROM customer c

LEFT JOIN salesorder so ON c.CustomerID = so.CustomerID

GROUP BY c.CustomerID, c.Name;

select * from vw_CustomerOrderSummary;

```



	CustomerID	CustomerName	TotalOrders	TotalAmountSpent	LastOrderDate
1	1	Pavla	0	NULL	NULL
2	2	Daune	1	29549.27	2008-12-11 11:36:26
3	3	Nevile	2	99452.77	2010-11-30 13:33:08
4	4	Eustace	1	29705.50	2019-12-22 19:57:18
5	5	Elga	0	NULL	NULL

Query executed successfully.

--Q10. Write a stored procedure sp_GetSupplierSales that takes a SupplierID as input and returns the total sales amount for all products supplied by that supplier.

```

CREATE OR ALTER PROCEDURE sp_GetSupplierSales

    @SupplierID INT

AS

BEGIN

    SET NOCOUNT ON;

    SELECT

        s.SupplierID,

        s.Name AS SupplierName,

        ISNULL(SUM(sod.TotalAmount), 0) AS TotalSalesAmount

    FROM supplier s

```



```
JOIN purchaseorder po ON s.SupplierID = po.SupplierID
```

```
JOIN purchaseorderdetail pod ON po.OrderID = pod.OrderID
```

```
JOIN product p ON pod.ProductID = p.ProductID
```

```
-- Join onto sales order details to find out what actually sold to final customers
```

```
JOIN salesorderdetail sod ON p.ProductID = sod.ProductID
```

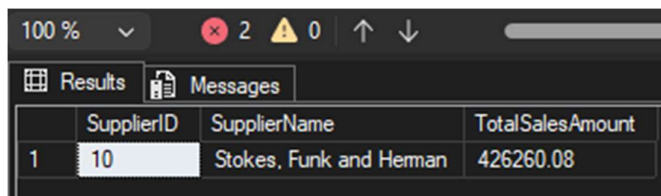
```
WHERE s.SupplierID = @SupplierID
```

```
GROUP BY s.SupplierID, s.Name;
```

```
END;
```

```
DECLARE @ID INT = 10; -- Yahan apni marzi ki ID likhein
```

```
EXEC sp_GetSupplierSales @SupplierID = @ID;
```



The screenshot shows a SQL Server query results window. At the top, there is a toolbar with a zoom level of 100%, a status bar indicating 2 errors and 0 warnings, and navigation arrows. Below the toolbar are two tabs: 'Results' (active) and 'Messages'. The 'Results' tab displays a table with the following data:

	SupplierID	SupplierName	TotalSalesAmount
1	10	Stokes, Funk and Herman	426260.08